

第五章 运算方法与运算器(三)

秦磊华 计算机学院

基于补码数据表示研究运算方法和设计运算器(简)

5.3 定点乘法运算

5.4 定点除法运算

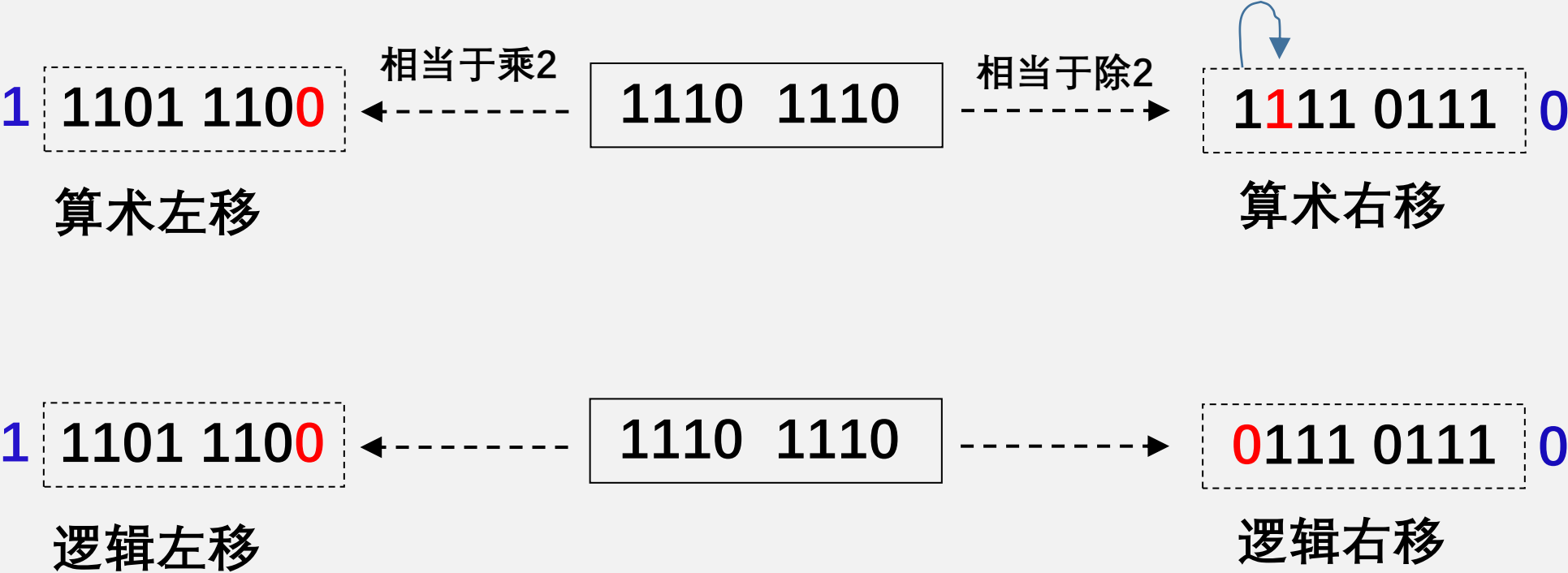
CONTENT





5.3 定点乘法运算

1. 移位操作





5.3 定点乘法运算

2. 二进制乘法的手工过程

×

1 0 1 0 (X)

1 0 1 1 (Y)

1 0 1 0 (0)

1 0 1 0 (1)

0 0 0 0 (2)

+ 1 0 1 0 (3)

1 1 0 1 1 1 0

↓

- ◆ 乘法可由加法实现 (!)
- ◆ 每次加数左移位数不同
- ◆ 每次加数的值要么为0，要么为X，取决于对应的乘数Y_n
- ◆ 需要长度为2n的积寄存器



5.3 定点乘法运算

2. 二进制乘法的手工过程

$$\begin{array}{r} 1010 \\ \times 1011 \\ \hline 1010 \\ 0000 \\ + 1010 \\ \hline 1101110 \end{array}$$

- 1) 乘法可由加法实现 (!)
- ◆ 可直接由前面基于FA设计的运算器来实现吗?
 - ◆ 如何解决?

 循环累加

2. 二进制乘法的手工过程

$$\begin{array}{r}
 1010 \\
 \times 1011 \\
 \hline
 1010 \quad \leftarrow (0) \\
 1010 \quad \leftarrow (1) \\
 0000 \quad \leftarrow (2) \\
 + 1010 \quad \leftarrow (3) \\
 \hline
 1101110
 \end{array}$$

2)每次加数左移位数不同

◆ 为什么每次加数要左移且位数不同？

◆ 如何解决？

每次运算完成后部分积右移1位，
实现累积右移效果



5.3 定点乘法运算

2. 二进制乘法的手工过程

1 0 1 0

×

1 0 1 1

1 0 1 0

0 1 0 1 0

+

1 0 1 0

1 1 1 1

0 1 1 1 1 0

+

0 0 0 0

0 1 1 1 1 0

0 0 1 1 1 1 0

+

1 0 1 0

1 1 0 1 1 1 0

0 1 1 0 1 1 1 0

1 0 1 0

×

1 0 1 1

1 0 1 0

1 0 1 0

0 0 0 0

+

1 0 1 0

1 1 0 1 1 1 0

7



5.3 定点乘法运算

2. 二进制乘法的手工过程

	1 0 1 0	
×	1 0 1 1	
	1 0 1 0	
→	0 1 0 1	0
+	1 0 1 0	
	1 1 1 1	
→	0 1 1 1	1 0
+	0 0 0 0	
	0 1 1 1	1 0
→	0 0 1 1	1 1 0
+	1 0 1 0	
	1 1 0 1	1 1 0
→	0 1 1 0	1 1 1 0

3) 需要长度为2n的积寄存器

◆ 长度为2n的积是逐步形成的

◆ 长度为2n的积由2部分构成



如何构成？

5.3 定点乘法运算

3. 原码一位乘法算法

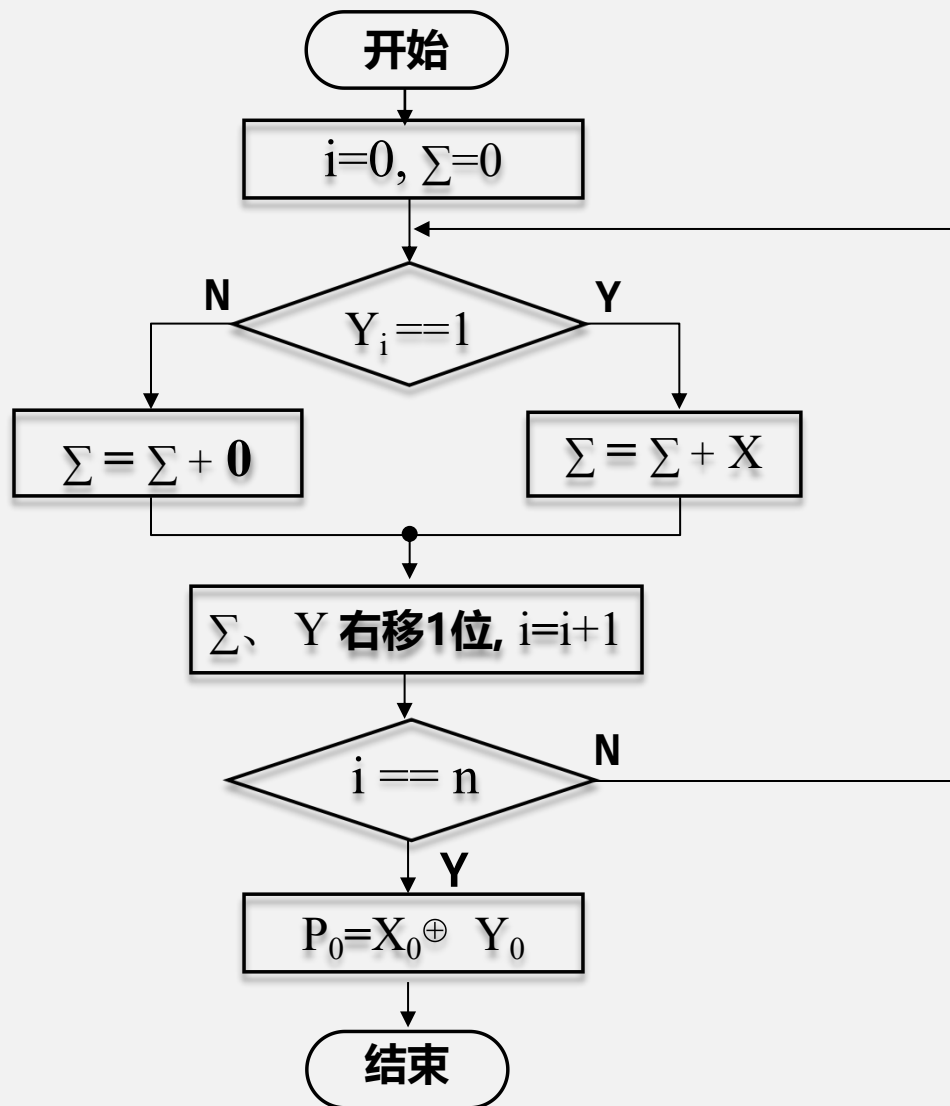
(1) 符号单独运算：直接异或

(2) 绝对值相乘

仅需考虑数值部分的计算

(3) n 次加法， n 次右移

(4) 部分积右移包含进位位



5.3 定点乘法运算

3. 原码一位乘法算法

例1 $x=+ 1101, y=-11$

求 $[X]_{原} \times [Y]_{原}$

解: $[X]_{原} = 01101$

$[Y]_{原} = 10011$

(1) 符号单独运算: 直接异或

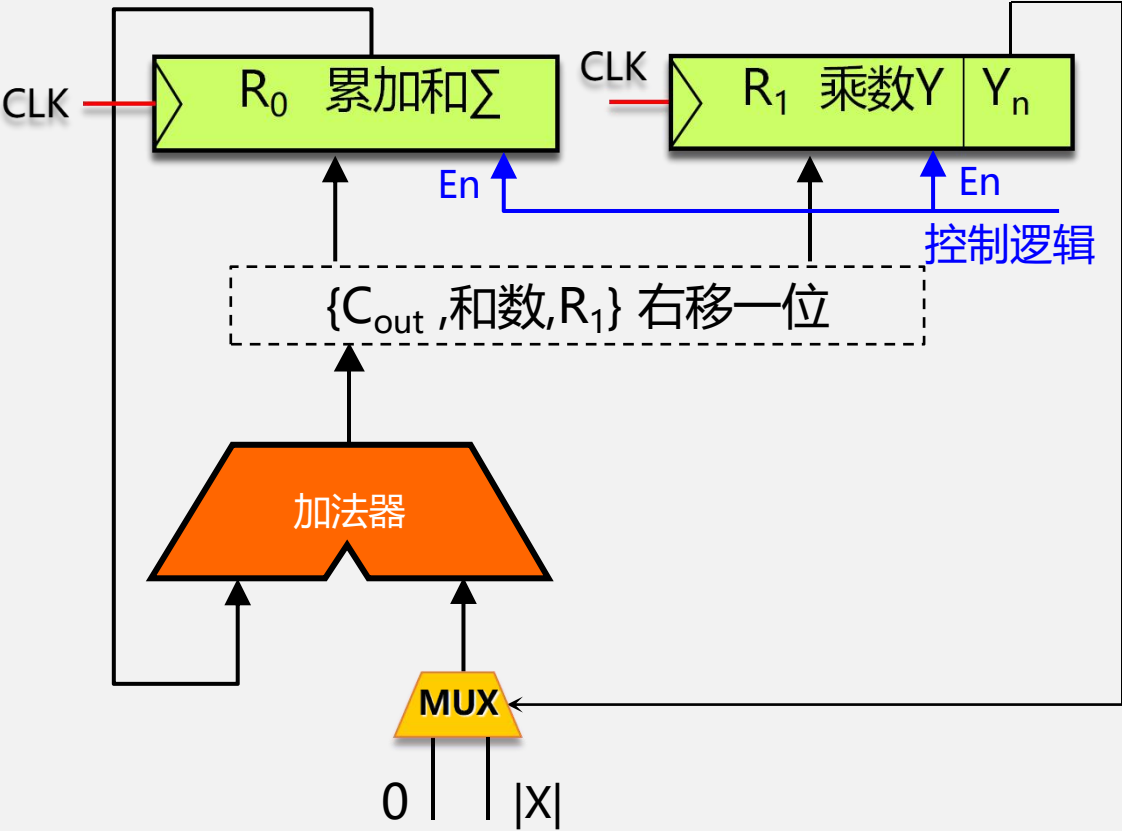
(2) 绝对值相乘

仅需考虑数值部分的计算

$[X]_{原} \times [Y]_{原} = 1\ 0010\ 0111$

$\Sigma=0$	0 0 0 0 0 0		乘数判断位 Y_n 0 0 0 1 <u>1</u>
	+ 0 0 1 1 0 1		
	0 0 1 1 0 1		
	→ 0 0 0 1 1 0	1 (?)	1 0 0 0 <u>1</u>
	+ 0 0 1 1 0 1		
	0 1 0 0 1 1	1	
	→ 0 0 1 0 0 1	1 1	1 1 0 0 <u>0</u>
	+ 0 0 0 0 0 0		
	0 0 1 0 0 1	1 1	
	→ 0 0 0 1 0 0	1 1 1	1 1 1 0 <u>0</u>
	+ 0 0 0 0 0 0		
	0 0 0 1 0 0	1 1 1	
	→ 0 0 0 0 1 0	0 1 1 1	0 1 1 1 <u>0</u>

4. 原码一位乘法硬件实现



- R_0 存放部分积高 n 位，初值为 0
- R_1 存放 Y 、 Σ 其它位（如何载入 Y 初值）
- 运算结果右移后送寄存器输入！
- 时钟到来， R_0 ， R_1 锁存新值
- 状态机控制使能信号停机
- 停机后乘积存放在 R_0 ， R_1 中

$$\{\Sigma, Y\} = \{\Sigma + Y_n|X|, Y\} / 2 \quad \text{逻辑右移}$$



5.3 定点乘法运算

5.补码booth一位乘法

1) 被乘数X符号任意，乘数Y为正

$$[X]_{\text{补}} = X_0.X_1X_2\cdots X_n \quad [Y]_{\text{补}} = 0.Y_1Y_2\cdots Y_n$$

$$[X]_{\text{补}} \times [Y]_{\text{补}} = (2 + X) \times Y = (2^{n+1} + X) \times Y \quad (\text{MOD } 2)$$

$$= 2^{n+1}Y + XY$$

$$= 2 \times 2^n \times 0.Y_1Y_2\cdots Y_n + XY$$

$$= 2(Y_1Y_2\cdots Y_n) + XY$$

$$= 2 + XY \quad (\text{MOD } 2)$$

$$= [XY]_{\text{补}} = [X]_{\text{补}} \times Y \quad (1)$$

5.3 定点乘法运算

5.补码booth一位乘法

2) 被乘数[X]符号任意，乘数[Y]为负数

$$[X]_{\text{补}} = X_0.X_1X_2\cdots X_n \quad [Y]_{\text{补}} = 1.Y_1Y_2\cdots Y_n$$

$$[Y]_{\text{补}} = 2 + Y \quad \Longrightarrow \quad Y = [Y]_{\text{补}} - 2$$

$$Y = [Y]_{\text{补}} - 2 = 1 + 0.Y_1Y_2\cdots Y_n - 2 = 0.Y_1Y_2\cdots Y_n - 1$$

$$[X \times Y]_{\text{补}} = [X \times (0.Y_1Y_2\cdots Y_n - 1)]_{\text{补}}$$

$$= [X \times 0.Y_1Y_2\cdots Y_n - X]_{\text{补}}$$

$$= [X \times 0.Y_1Y_2\cdots Y_n]_{\text{补}} - [X]_{\text{补}}$$

$$= [X]_{\text{补}} \times 0.Y_1Y_2\cdots Y_n - [X]_{\text{补}}$$

$$= [X]_{\text{补}} \times 0.Y_1Y_2\cdots Y_n - Y_0[X]_{\text{补}}$$

当 Y 为正数时， $Y_0=0$ ，因此，该式也包含了(1)



5.3 定点乘法运算

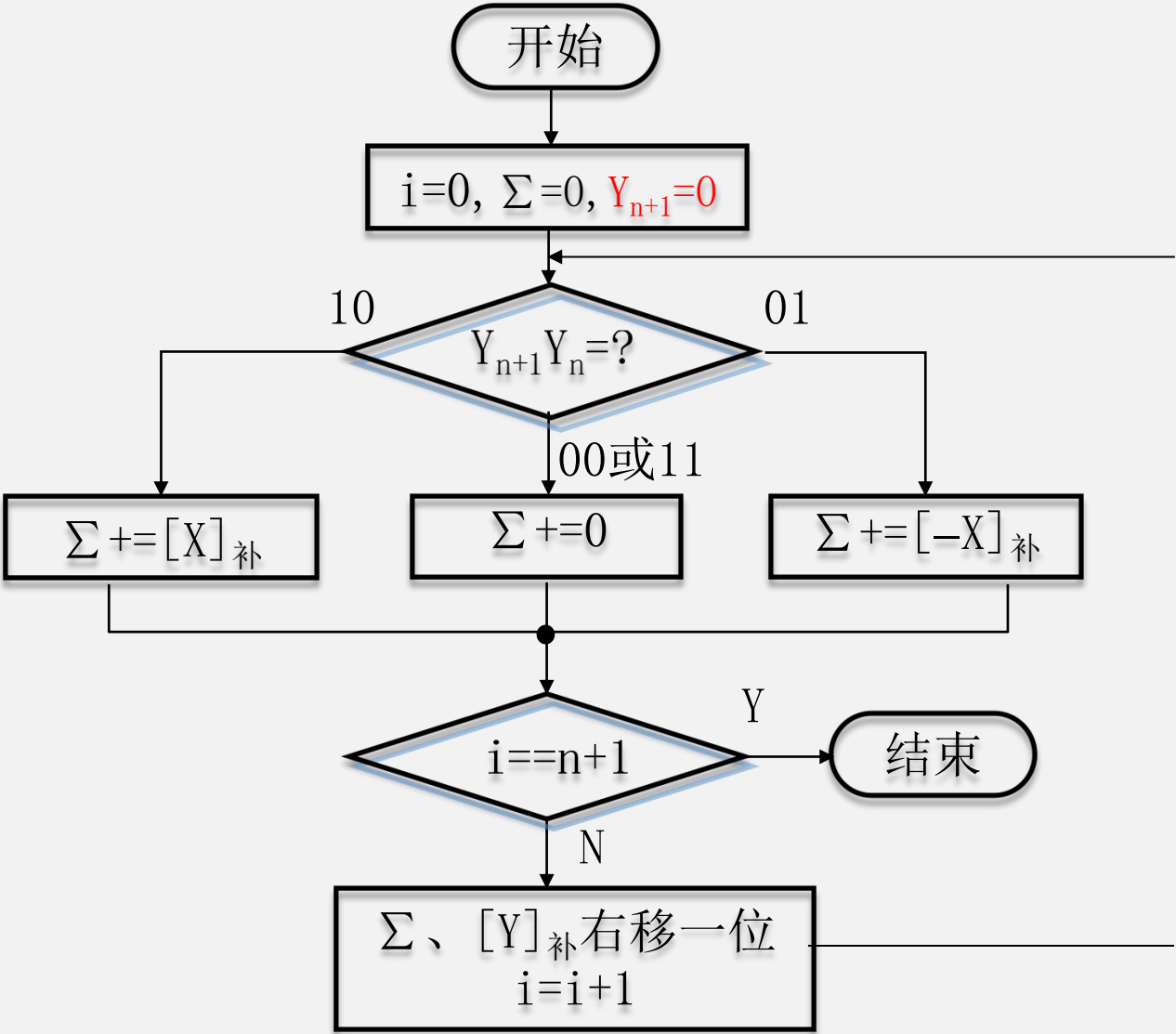
5.补码booth一位乘法

$$\begin{aligned}[X \times Y]_{\text{补}} &= [X]_{\text{补}} \times 0.Y_1 Y_2 \cdots Y_n - Y_0 [X]_{\text{补}} \\&= [X]_{\text{补}} \times (-Y_0 + 0.Y_1 Y_2 \cdots Y_n) \\&= [X]_{\text{补}} \times (-Y_0 + Y_1 2^{-1} + Y_2 2^{-2} + \cdots Y_n 2^{-n}) \\&= [X]_{\text{补}} \times [-Y_0 + (Y_1 - Y_1 2^{-1}) + (Y_2 2^{-1} - Y_2 2^{-2}) + \cdots (Y_n 2^{-n+1} - Y_n 2^{-n})] \\&= [X]_{\text{补}} \times [(Y_1 - Y_0) + (Y_2 - Y_1) 2^{-1} + (Y_3 - Y_2) 2^{-2} + \cdots (-Y_n) 2^{-n}] \\&= [X]_{\text{补}} \times [(Y_1 - Y_0) + (Y_2 - Y_1) 2^{-1} + (Y_3 - Y_2) 2^{-2} + \cdots (0 - Y_n) 2^{-n}]\end{aligned}$$

└ $Y_{n+1} = 0$

5.补码booth一位乘法

- 加法次数: $n+1$ 次
- 算术右移次数为 n 次
- 符号位参与运算





5.3 定点乘法运算

5.补码booth一位乘法

例2 $x=+1101, y=-11$

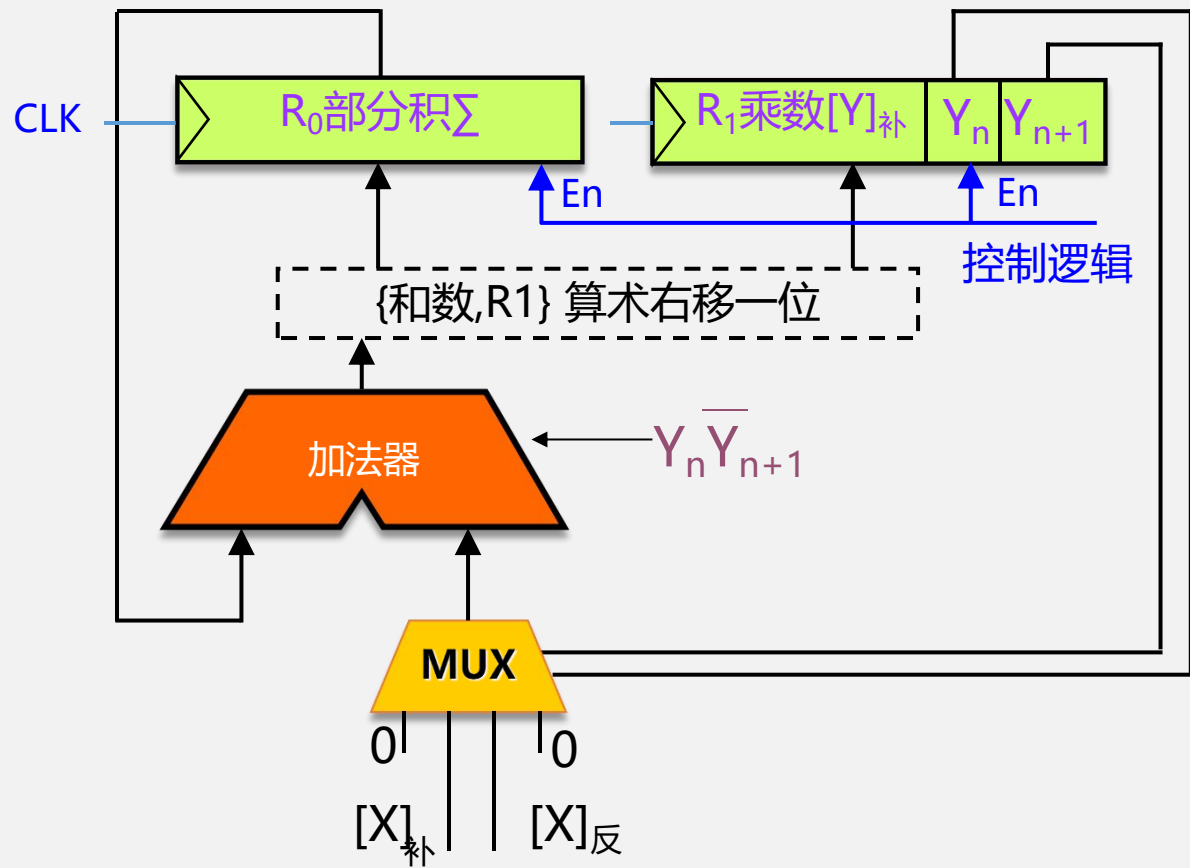
求 $[X]_{补} \times [Y]_{补}$

解： $[X]_{补}=01101, [-X]_{补}=10011$

$[Y]_{补}=11101$

$[X]_{补} \times [Y]_{补} = 1\ 110\ 11001$

$\Sigma=0$	00	0000		乘数判断位 Y_nY_{n+1}
	+11	0011		1110 <u>10</u>
	11	0011		
$\rightarrow$	11	1001	1	1111 <u>01</u>
	+00	1101		
	00	0110	1	
$\rightarrow$	00	0011	01	0111 <u>10</u>
	+11	0011		
	11	0110	01	
$\rightarrow$	11	1011	001	0011 <u>11</u>
	+00	0000		
	11	1011	001	
$\rightarrow$	11	1101	1001	1001 <u>11</u>
	+00	0000		
	11	1101	1001	

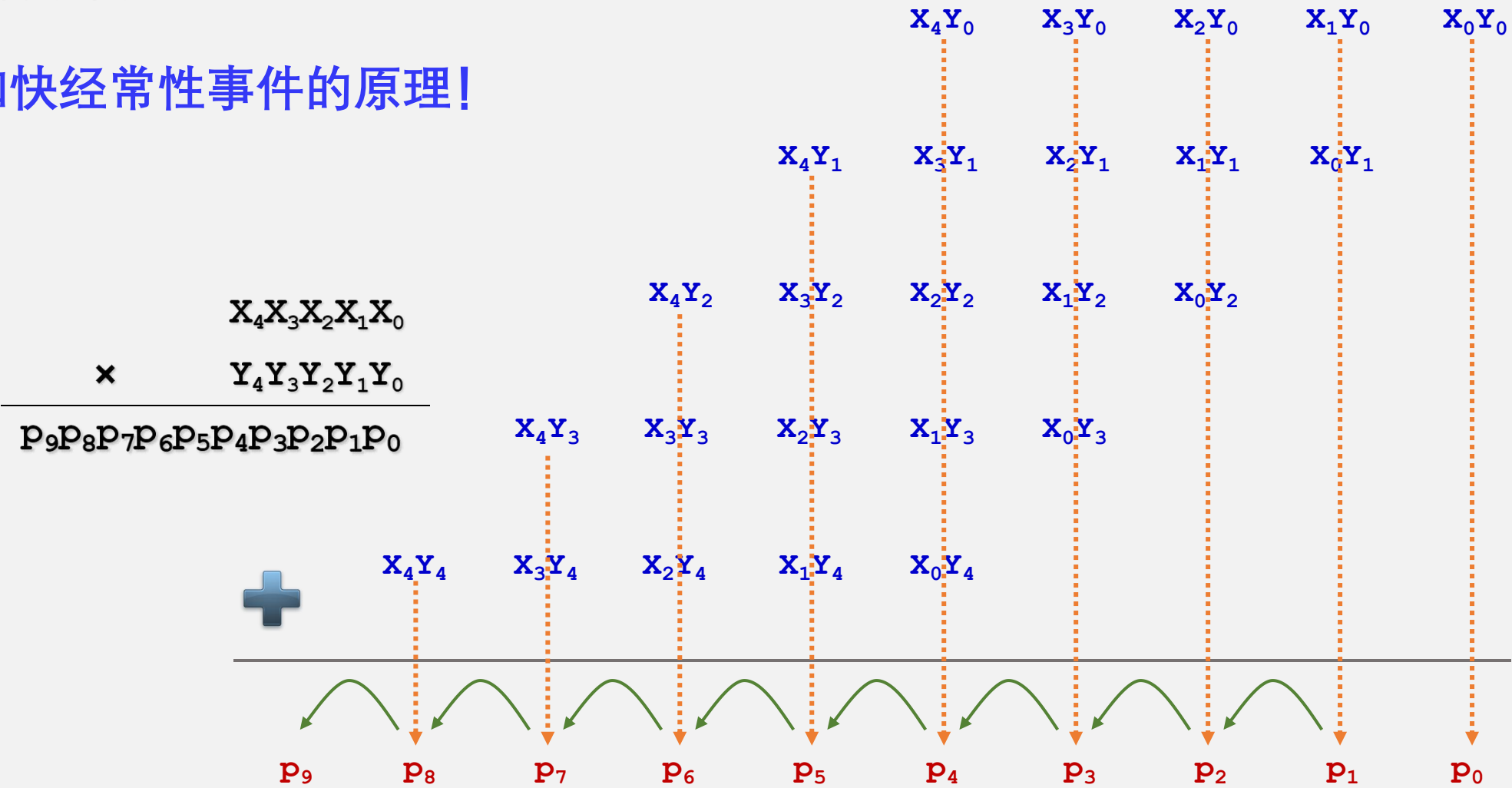




5.3 定点乘法运算

7.阵列乘法器

加快经常性事件的原理！

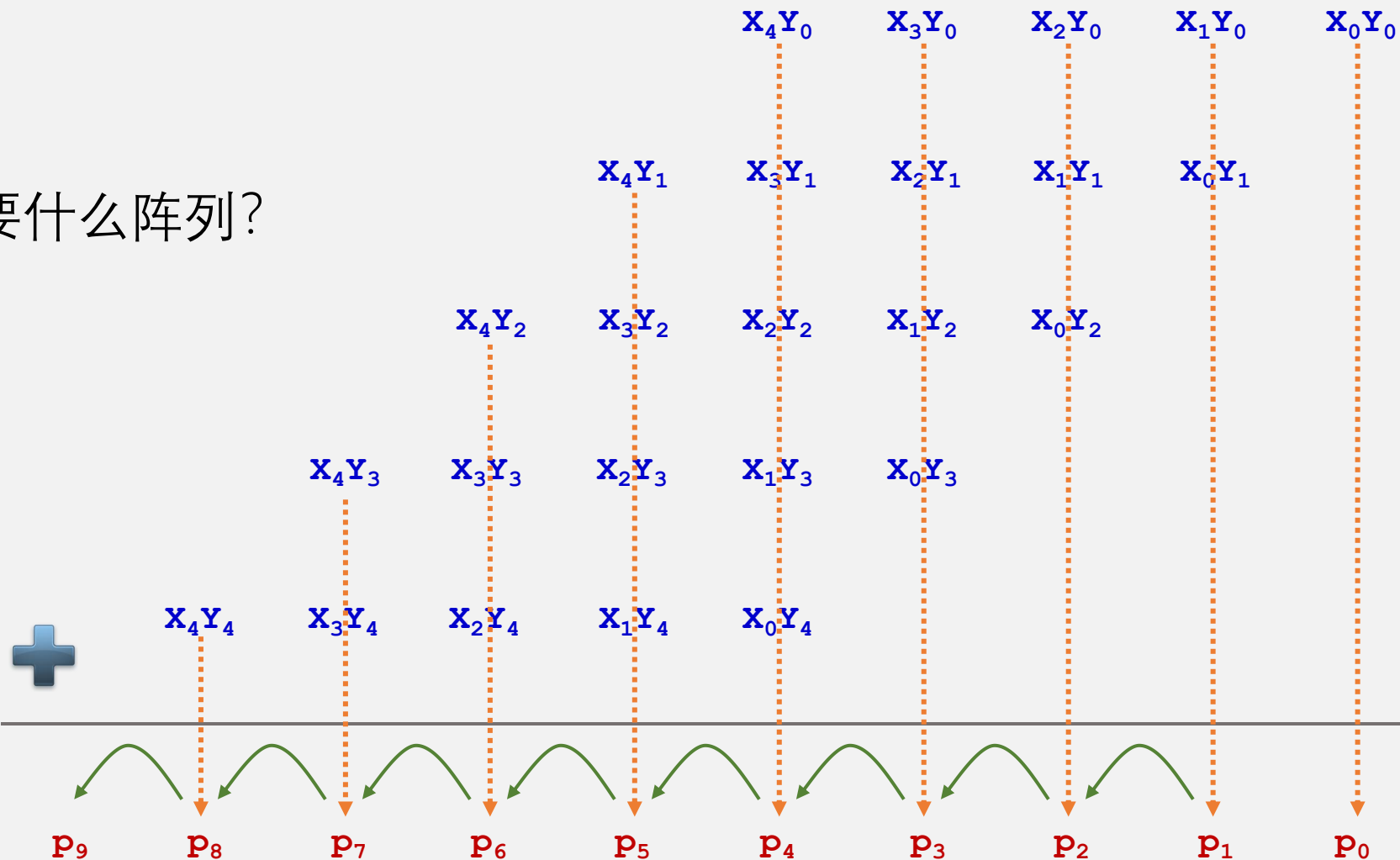




5.3 定点乘法运算

7. 阵列乘法器

- ◆ 阵列?
- ◆ 实现右边乘法需要什么阵列?





5.3 定点乘法运算

7. 阵列乘法器

1) 与门阵列

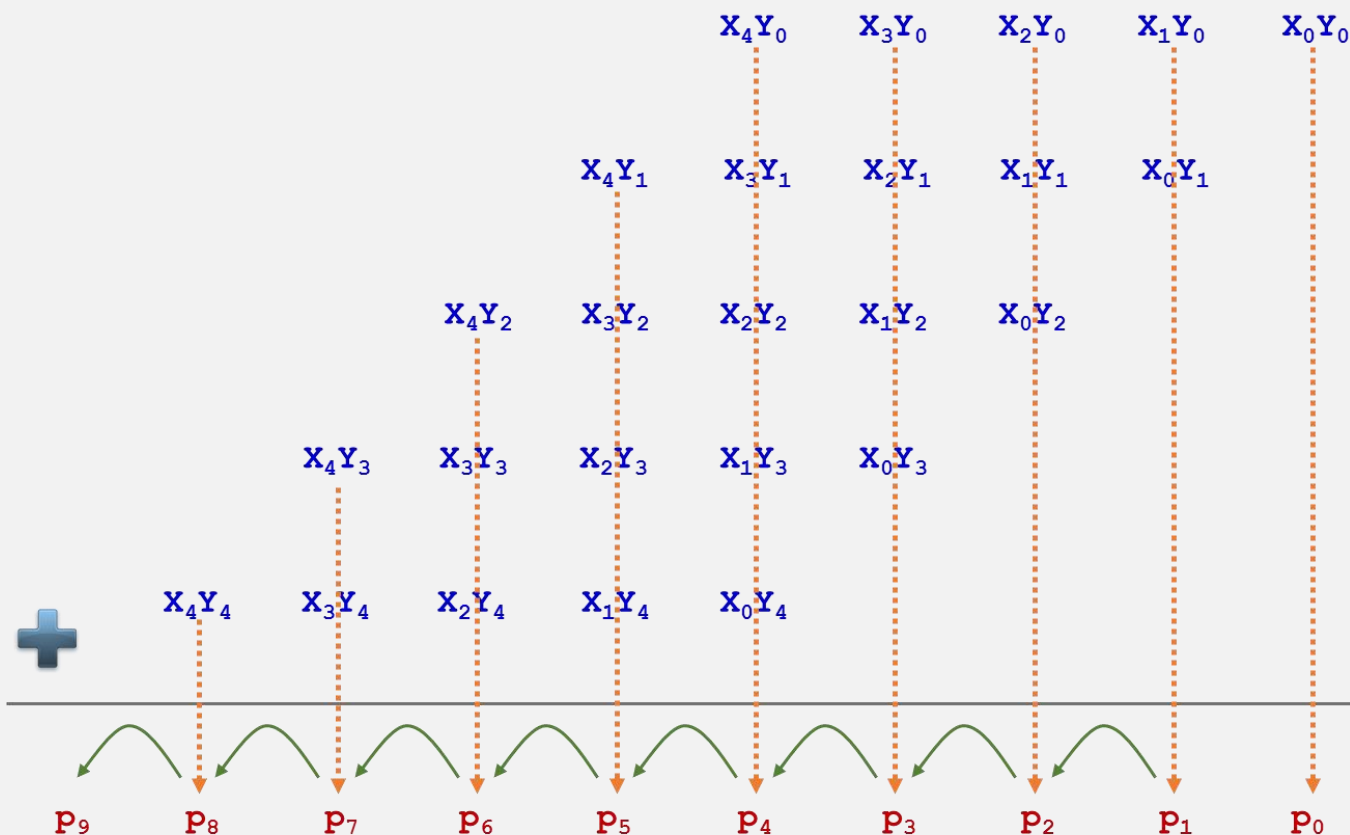
$$R = X * Y$$

$$1 \times 1 = 1 \quad = 1.1$$

$$1 \times 0 = 0 \quad = 1.0$$

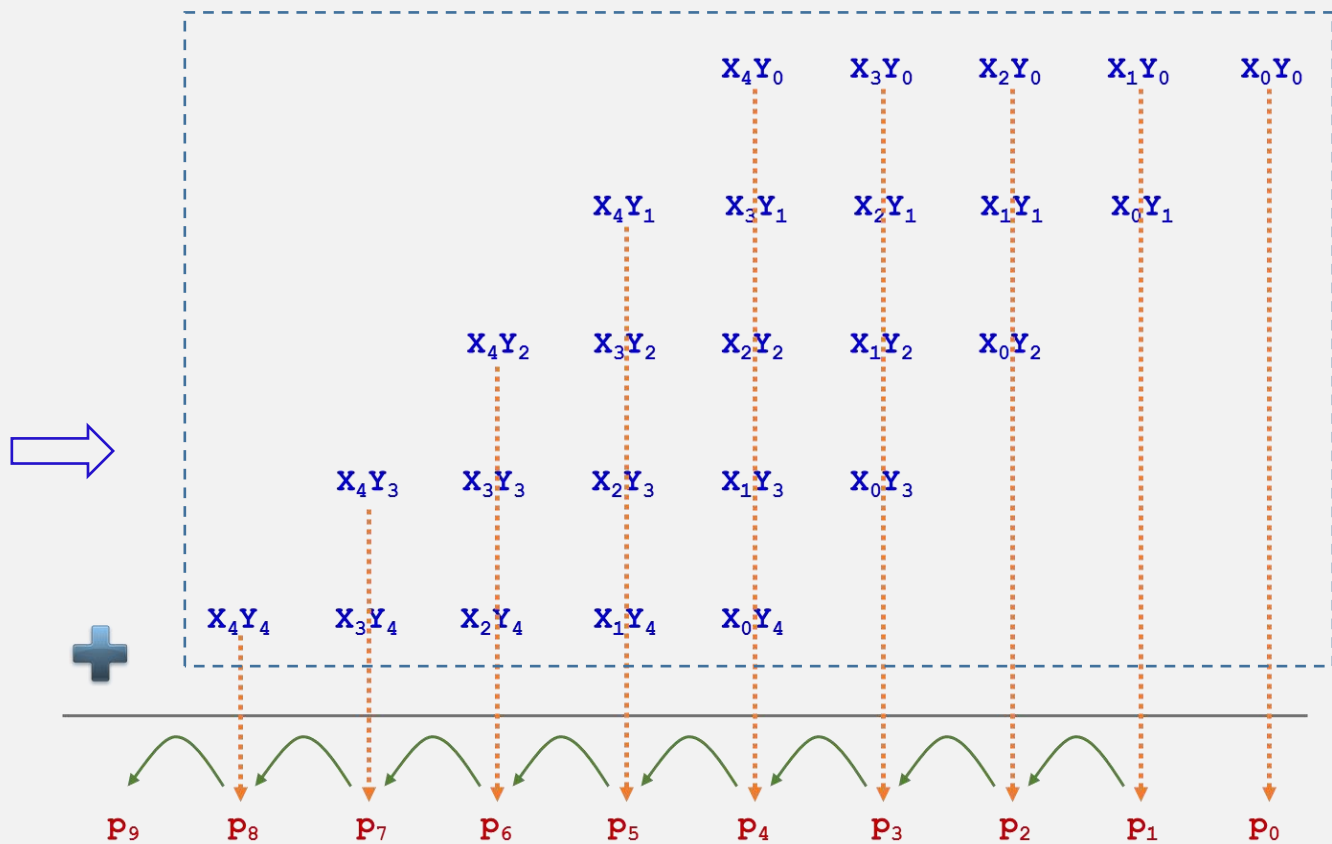
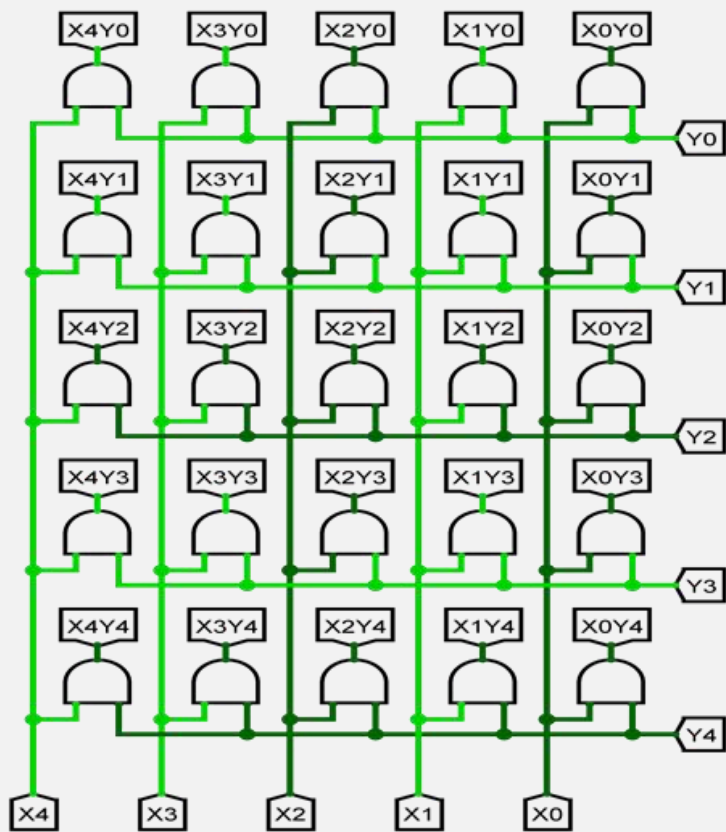
$$0 \times 1 = 0 \quad = 0.1$$

$$0 \times 0 = 0 \quad = 0.0$$



5.3 定点乘法运算

7. 阵列乘法器

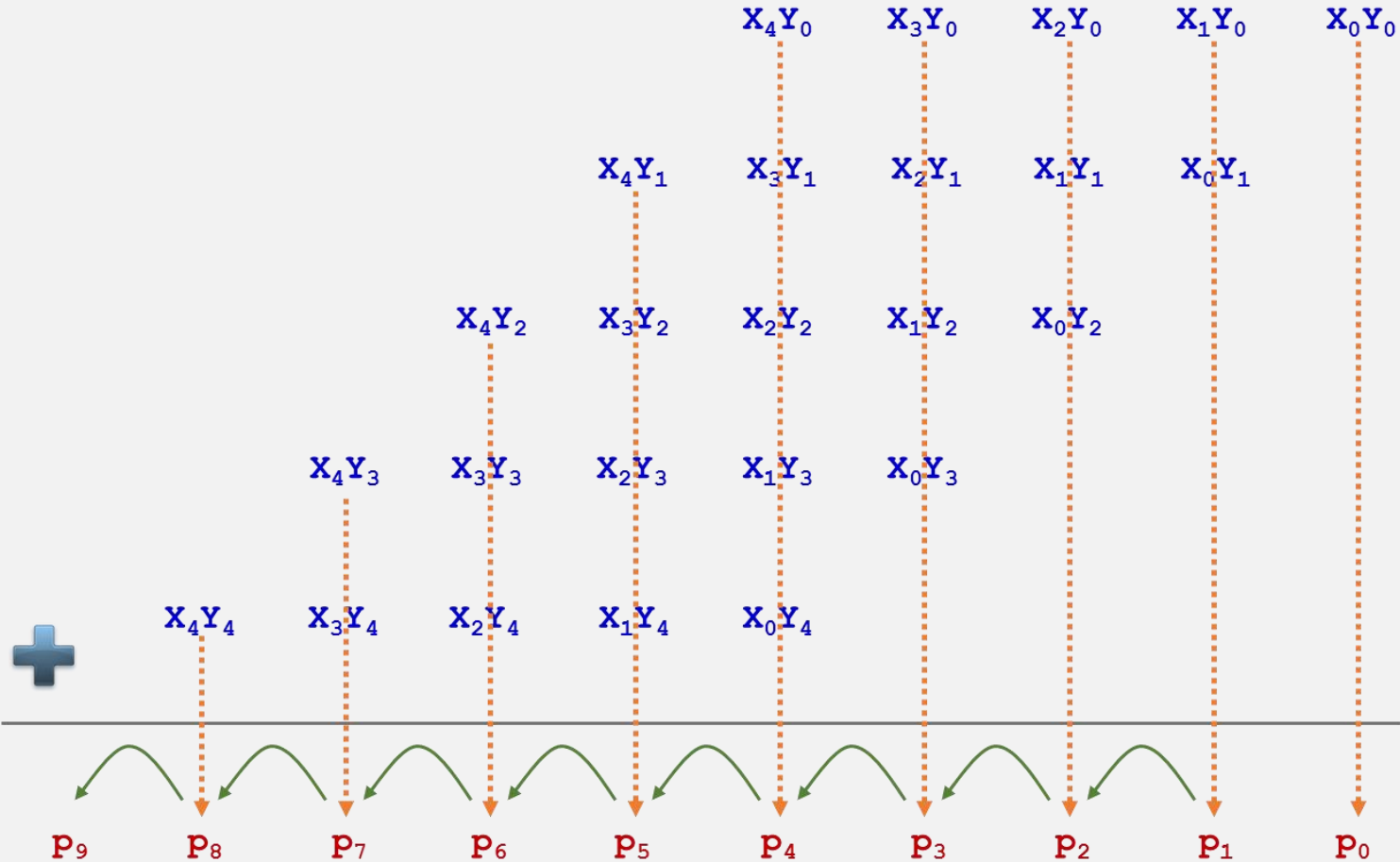




5.3 定点乘法运算

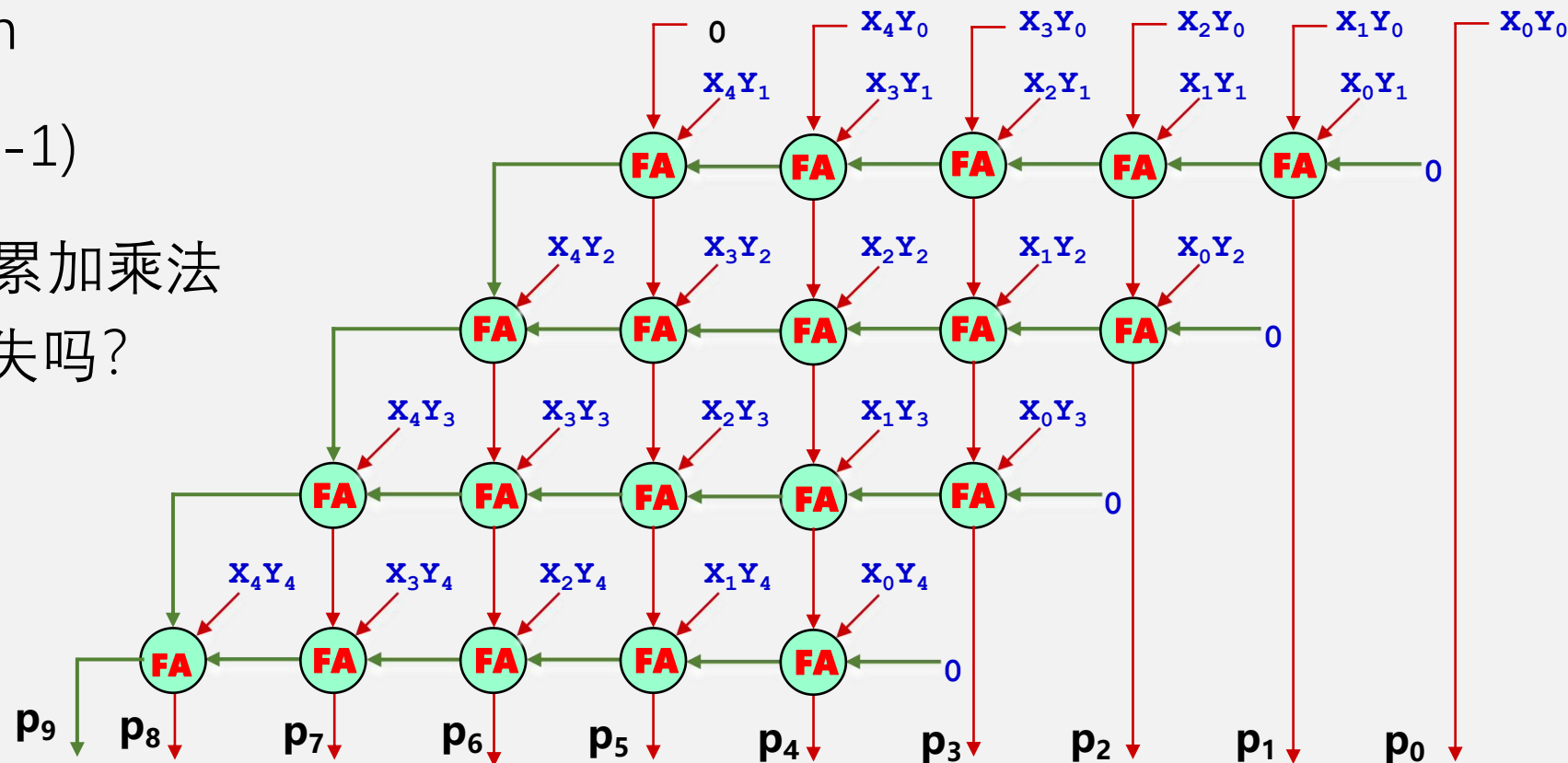
7. 阵列乘法器

- 1) 与门阵列
- 2) 还需要什么阵列?



7. 阵列乘法器

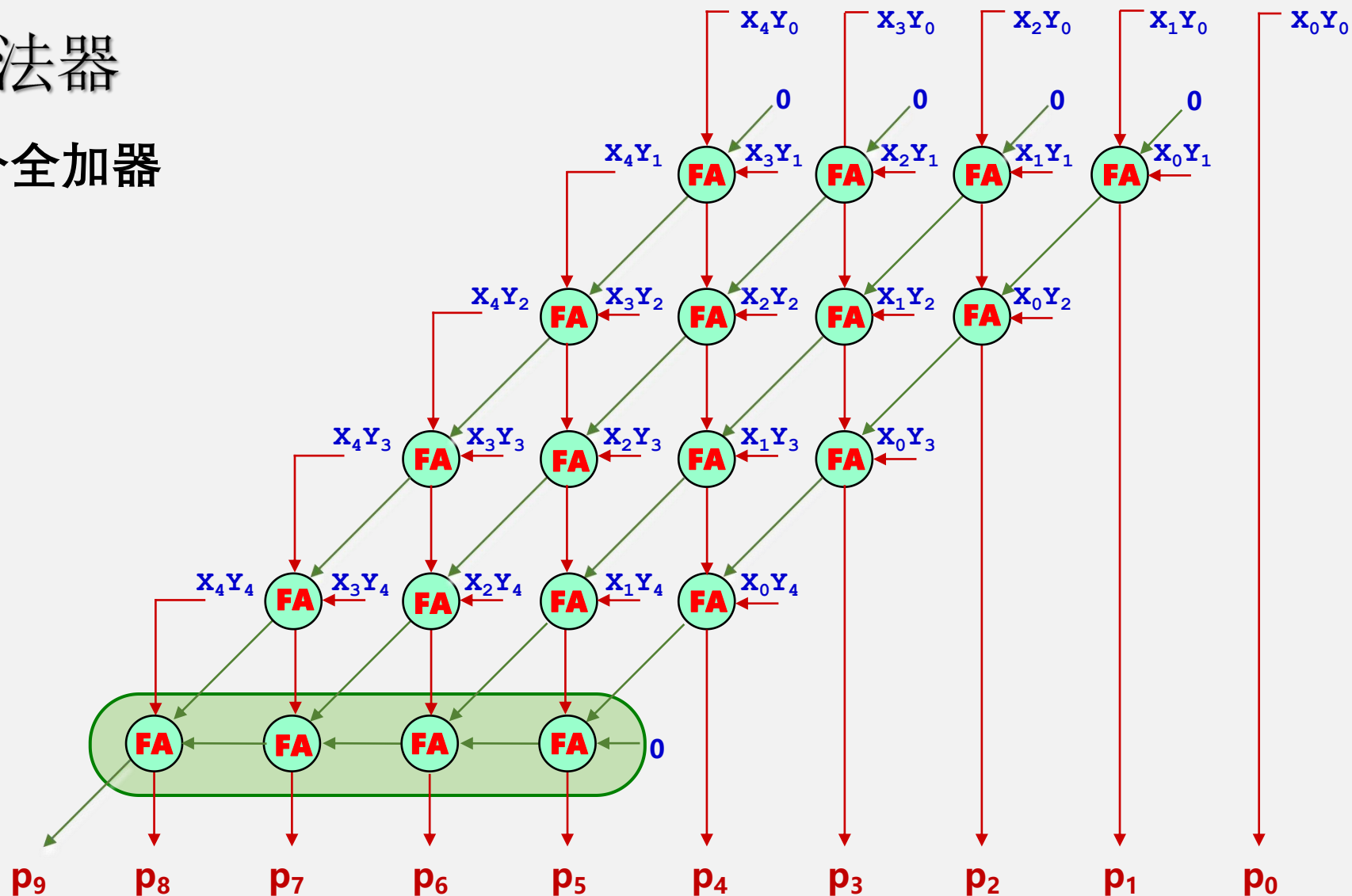
- 1) 与门阵列: $n \times n$
- 2) FA阵列: $n \times (n-1)$
- 3) 能缓解循环累加乘法带来的性能损失吗?



5.3 定点乘法运算

7. 阵列乘法器

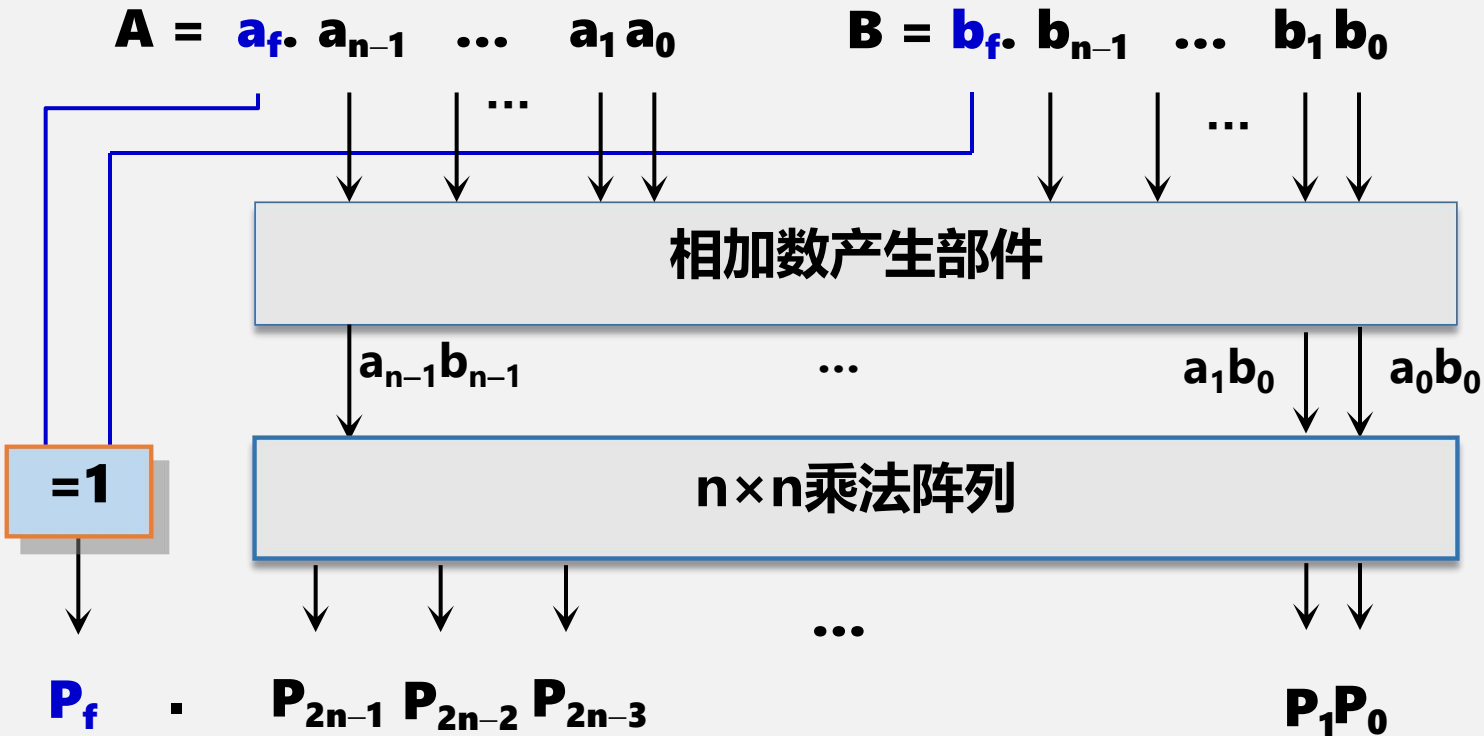
◆ $n*(n-1)$ 个全加器





5.3 定点乘法运算

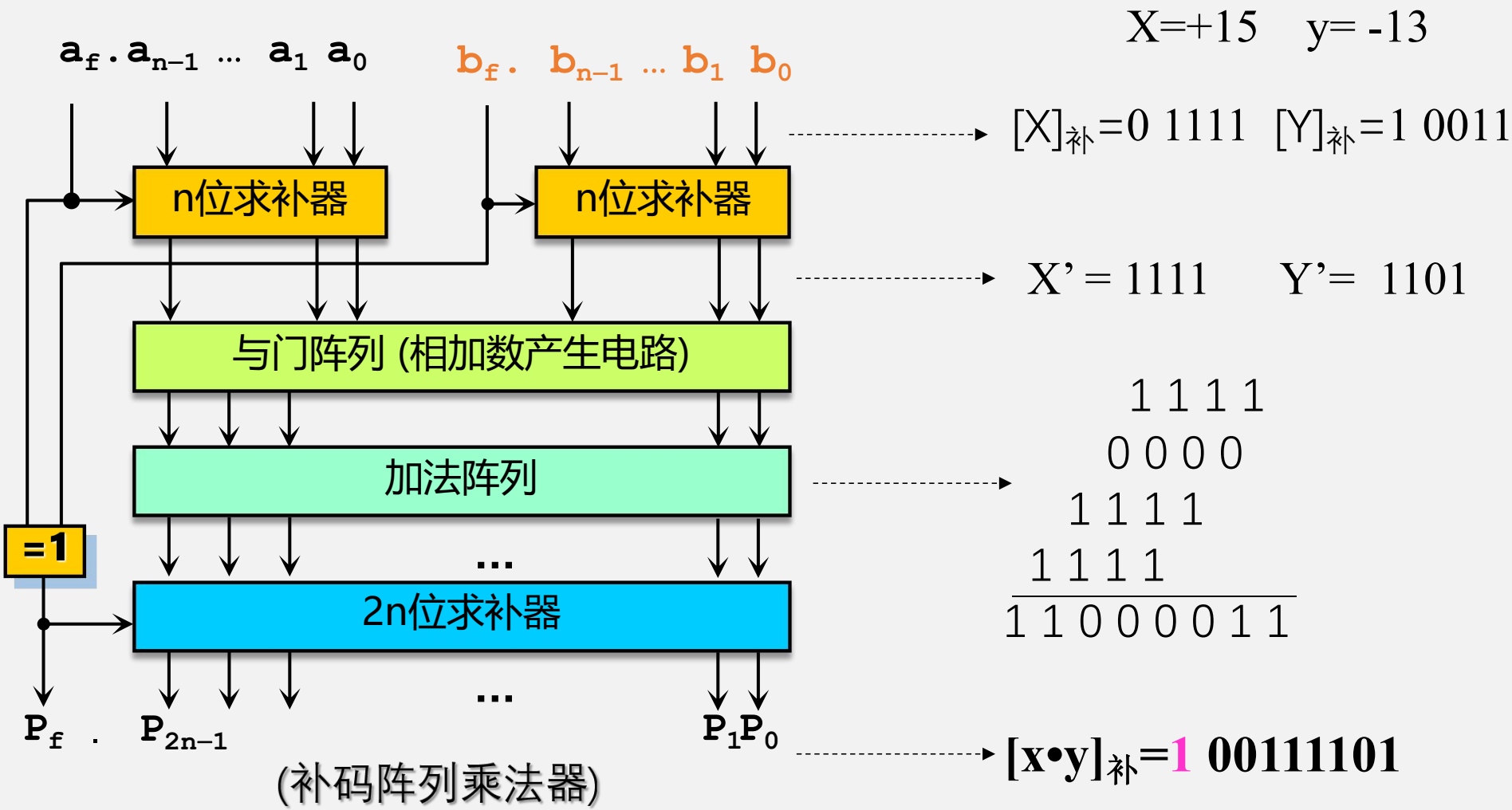
7. 阵列乘法器



(原码阵列乘法器)

5.3 定点乘法运算

7. 阵列乘法器



8. 高级语言中的整数乘法及溢出判断

高级语言中两个 n 位整数相乘得到的结果通常也是一个 n 位整数，结果只取 $2n$ 位乘积中的低 n 位。

```
int mul(int x, int y)
{
    int z=x*y;
    return z;
}
```

如何判断 z 是否溢出？



8. 高级语言中的整数乘法及溢出判断

https://blog.csdn.net/weixin_31092009/article/details/117121295

```
/* Determine whether arguments can be multiplied without overflow.
*/
int tmul_ok(int x, int y)
{undefined
#if 0
int p = x * y;
return !x || p/x==y;
#endif
return umul_ok(x, y); /* 直接调用 */
}
```

9. 汇编语言中的整数乘法及溢出判断

- ◆ **硬件不判溢出**，仅保留 $2n$ 位乘积，供软件使用
- ◆ 程序不判断溢出，编译器也不生成用于溢出处理的代码，会发生整数溢出问题。
- ◆ 乘法指令的操作数长度为 n ，而乘积长度为 $2n$ ：

IA-32:

16位乘积结果存放在AX (8*8)

32位乘积结果存放在DX-AX (16*16)

64位乘积或EDX-EAX (32*32)

MIPS: 32位带符号整数相乘，64位乘积置于两个32位内部寄存器Hi和Lo中，可根据Hi寄存器中的每一位是否等于Lo寄存器中的第一位来进行溢出判断。

<https://www.cnblogs.com/os2os/p/6565252.html>

1996年6月4日，阿丽亚娜5型运载火箭（Ariane 5）在法国库鲁的欧洲运载火箭发射场发射，37秒后火箭解体并爆炸。火箭的开发费用大约70亿美元，火箭本体及运载的设备价值约5亿美元。两周后的调查报告指出，事故原因来自将火箭的水平速度的64位浮点数转换成16位整数时的溢出。

使用强类型语言时，当数值类型不相同，转换是必须的。从位数多的数据类型向少的转换有危险，但并不是一定出问题。

阿丽亚娜5型的转换，在4型时就在使用，但4型没有出问题。因为在决定做浮点数到整数的转换前，开发人员做过仔细调查，可以确定浮点数转换后的大小不会超过16位整数的范围，也就是说不会有溢出。但是，在开发5型时，这部分功能被直接拿来使用，而没有意识到5型火箭的水平速度产生的数值要比4型大很多，16位整数类型不能放下转换后的数值，这导致了灾难性的事故。

10. 高级语言中的变量与常量乘法

- ◆按照前述运算方法，整数乘法运算比移位和加法等运算所用时间长很多(!)
- ◆编译器在处理变量与常数相乘,如 $20 \times X$ 时，往往以移位、加法和减法的组合运算来代替乘法运算。

$x \times 20$ 转换为 $(x \ll 4) + (x \ll 2)$

一次乘法转换成了一次移4位、一次移2位和1次加法

$x \times 15$ 转换为 $(x \ll 4) - x$

- ◆移位加减组合运算和直接相乘结果一样（包括溢出）
- ◆是否优化取决于组合运算周期数是否小于乘法开销



5.4 定点除法运算

- 除法与加、减、乘法一样，也是一种常见的运算
- 除法运算包括定点整数除法与定点小数除法运算
- 对于字长为 n 位的除法，要得到字长为 n 位的商，要求被除数长度为 $2n$ 位，如何将字长为 n 位的除数变成长度为 $2n$ 位被除数，定点整数和定点小数的处理方法不同，需要关注。



5.4 定点除法运算

1. 手工除法

0 1 1 0 ← 商

1 0 1 1) 1 0 0 1 0 0 1

0 1 0 1 1

0 0 1 1 1 0

0 0 1 0 1 1

余数 0 0 0 0 1 1 1

- ◆ 除法可由减法实现
- ◆ Y每次右移次数不等
- ◆ 要判断两数是否够减 (?)
- 不够减, 商上0,
- 够减, 商上1, 求余数
- 除数右移一位
- 继续按规则上商直至所需位数

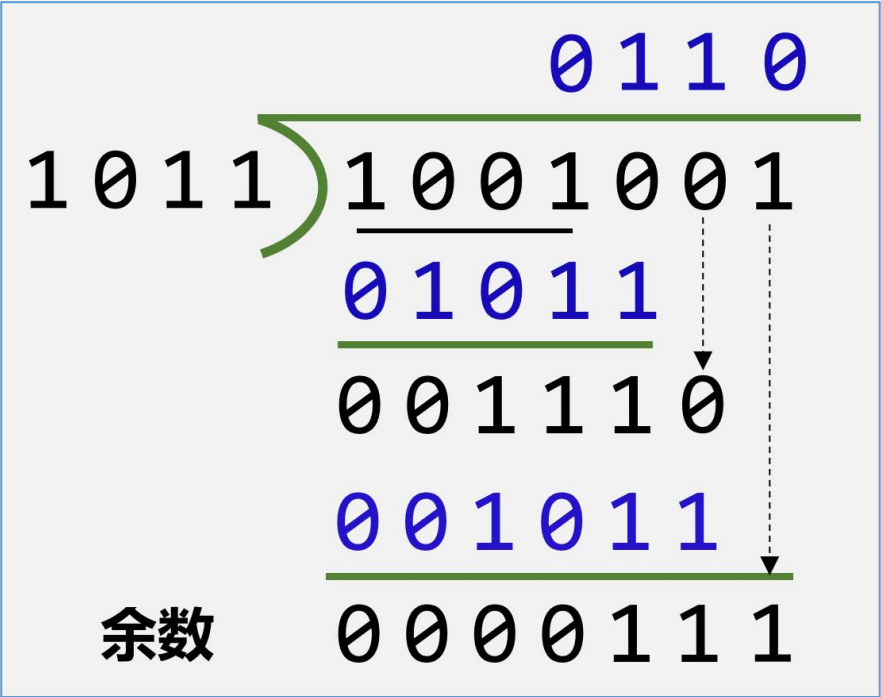


5.4 定点除法运算

2. 计算机中除法对手工除法的改进

1)对手工算法的改进

- ◆判断两数的大小用减法实现,
若差>0 ,则够除;反之不够除;
- ◆将手工每次右移除数,改为左移被除数.





5.4 定点除法运算

3. 原码恢复余数法除法

◆法则：

设 $[X]_{\text{原}} = X_f X_1 X_2 X_3 \cdots X_n$ ， $[Y]_{\text{原}} = Y_f Y_1 Y_2 Y_3 \cdots Y_n$ ，用原码一位除法求

$$Q = Q_0 Q_1 Q_2 Q_3 \cdots Q_n = X/Y$$

则： $Q_f = X_f \oplus Y_f$

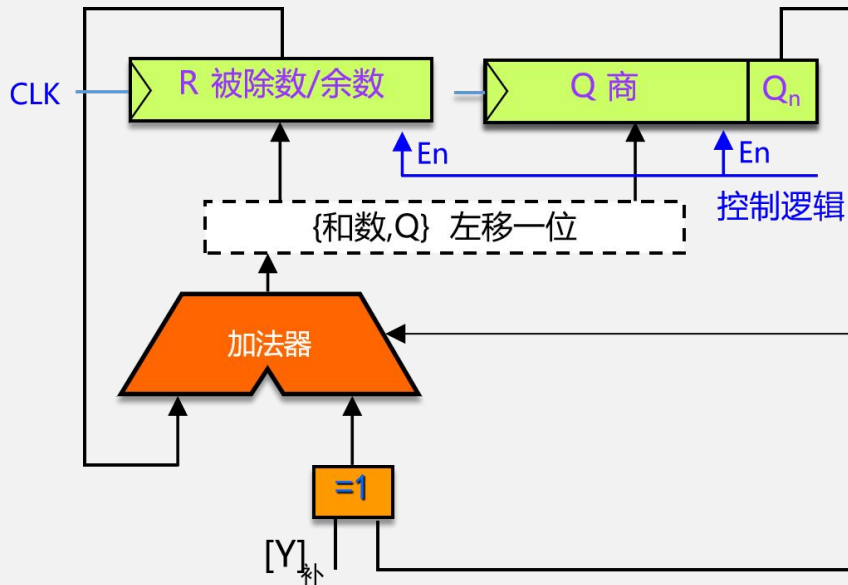
$$[Q]_{\text{原}} = (X_f \oplus Y_f).(0X_1 X_2 X_3 \cdots X_n (\text{需要将长度变为} 2n \text{位}) / 0Y_1 Y_2 Y_3 \cdots Y_n)$$

3. 原码恢复余数法除法

1) 特殊问题的处理

试商通过 $X - Y$ 进行

- ◆ 试商结果 > 0 时，商上 1；
- ◆ 试商的结果 < 0 时？？，
恢复余数！即 $+Y$ ，此时不能上商



2) 被除数扩展为2n位的方法

对定点小数除法而言， $2n$ 位的被除由被除数/余数+ 商寄存器构成(初始为0)

对定点整数除法而言， $2n$ 位的被除由被除数通过符号扩展而成， n 位符号扩展位存放在余数寄存器，原数据存放在商寄存器。

5.4 定点除法运算

3. 原码恢复余数法除法

例 $X=0.1001$, $Y= -0.1011$
 用原码一位除法求 $X \div Y$

解: $[X]_{\text{原}} = 0.1001$
 $[Y]_{\text{原}} = 1.1011$
 $[|X|]_{\text{补}} = 0.1001$
 $[|Y|]_{\text{补}} = 0.1011$
 $[-|Y|]_{\text{补}} = 1.0101$
 $Q=11101 \quad R= 00001*2^{-4} (!)$
 $\hookrightarrow 0 \oplus 1 = 1$

	被除数/余数R	上商位 Q_n (?)	说明
	00.1001	00000	X-Y试商
[-Y] +	11.0101		
	11.1110	00000 0	$R < 0$, 商上0, +Y恢复余数
	+ 00.1011		
	00.1001		
←	01.0010	00000 0	$R = 2R$, -Y试商
[-Y] +	11.0101		
	00.0111	00000 1	$R > 0$, $Q_n = 1$
←	00.1110	00001 1	$R = 2R$, -Y试商
[-Y] +	11.0101		
	00.0011	00001 1	$R > 0$, $Q_n = 1$
←	00.0110	00011 1	$R = 2R$, -Y试商
[-Y] +	11.0101		
	11.1011	00011 0	$R < 0$, $Q_n = 0$, +Y 恢复余数
	+ 00.1011		
	00.0110		
←	00.1100	00110	
[-Y] +	11.0101		
	00.0001	00110 1	$R > 0$, $Q_n = 1$
←		01101	



5.4 定点除法运算

3. 原码恢复余数法除法

- ◆需要进行恢复余数的操作
- ◆恢复余数的操作次数不确定，影响除法速度和控制。
- ◆实际应用通常采用不恢复余数除法/加减交替法。

4. 原码加减交替法除法

◆ 设某次除法运算余数为 $R_i > 0$ ，将 R_i 左移一位，然后减除数试商：

$$2R_i - Y$$

◆ 若试商结果小于0，则商上0，并恢复余数：

$$(2R_i - Y) + Y = 2R_i$$

◆ 再左移并试商：

$$2 * 2R_i - Y = 4R_i - Y \quad \dots\dots\dots (1)$$

◆ 若试商结果 $2R_i - Y < 0$ ，商上0，不恢复余数，而是直接左移一位并加Y：

$$2 * (2R_i - Y) + Y = 4R_i - Y \quad \dots\dots\dots (2)$$

5.4 定点除法运算

4. 原码加减交替法除法

例 $X=0.1001$, $Y= -0.1011$

用原码一位除法求 $X \div Y$

解: $[X]_{\text{原}}= 0.1001$

$[Y]_{\text{原}}= 1.1011$

$[|X|]_{\text{补}}= 0.1001$

$[|Y|]_{\text{补}}= 0.1011$

$[-|Y|]_{\text{补}}= 1.0101$

$Q=1.1101$ $R= 0.0001 \times 2^{-4}$

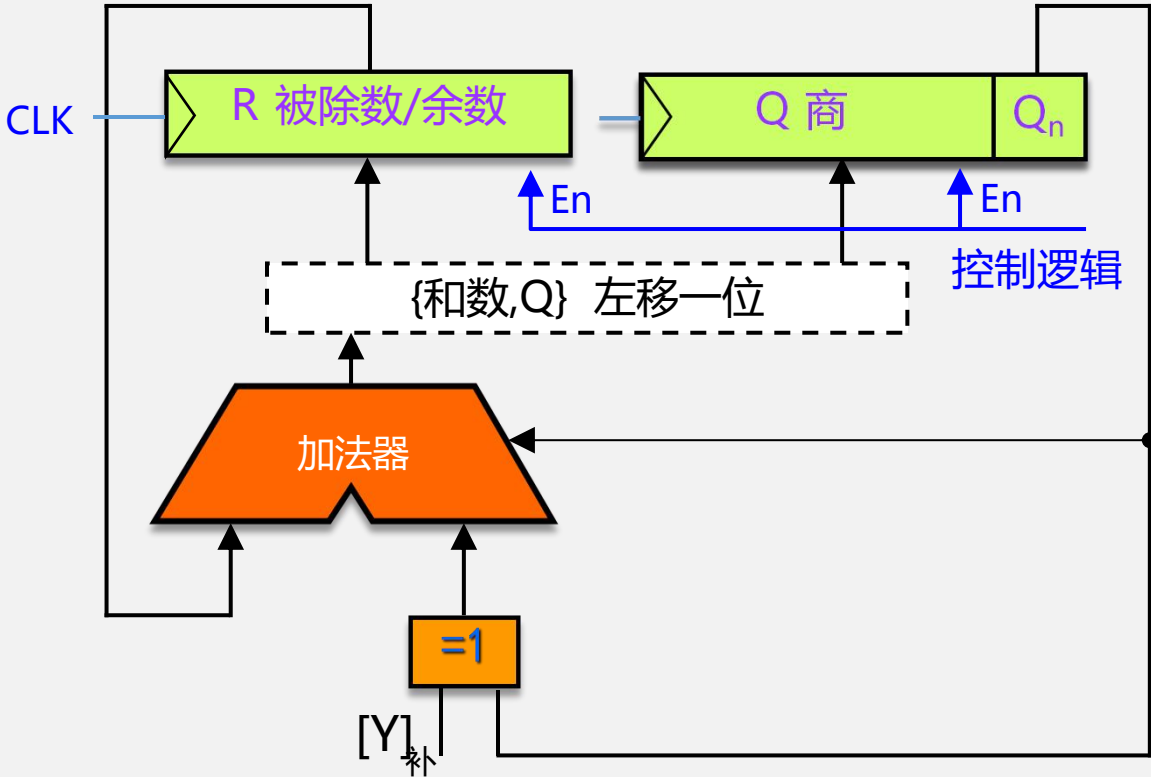
	被除数/余数R	上商位 Q_n (?)	说明
	00.1001	00000	X-Y试商
$[-Y]$ +	11.0101		
	11.1110	00000 0	$R < 0$, 商上0
	← 11.1100	00000 0	$R = 2R + Y$ (左移加除数)
$[Y]$ +	00.1011		
	00.0111	00000 1	$R > 0$, $Q_n = 1$
	← 00.1110	00001 1	$R = 2R - Y$
$[-Y]$ +	11.0101		
	00.0011	00001 1	$R > 0$, $Q_n = 1$
	← 00.0110	00011 1	$R = 2R - Y$
$[-Y]$ +	11.0101		
	11.1011	00011 0	$R < 0$, 商上0
	← 11.0110	00110 0	$R = 2R + Y$
$[Y]$ +	00.1011		
	00.0001	00110 1	$R > 0$, $Q_n = 1$
	← 00.0001	01101 1	

加法操作和左移操作均为N次，最后一次只移商



5.4 定点除法运算

4. 原码加减交替法除法(电路)



5. 补码加减交替法除法

1) 符号位参加运算

2) 试商方法不同于原码一位除法

- ◆ 回顾原码一位除法的试商-----减法实现

- ◆ 若采用原码试商方法存在的问题 (为什么不能直接减来试商)

3) 补码一位除法的试商及运算方法

- ◆ 被除数与除数同号，被除数减除数；反之加除数；(注意，这样得到的商是反码!)

- ◆ 余数与除数同号，商上1，余数左移一位，下次减除数；

反之，商上0，余数左移一位，下次加除数

- ◆ 重复上一步，包括符号位在内共执行 $n + 1$ 次，且最后一步只移商



5.4 定点除法运算

5. 补码加减交替法除法

4) 商与余数的修正

(1) 商的修正: 当商为负数时, 需要在得到的商第一位加“1”, 得到补码形式的商。

(2) 余数修正: 若余数与被除数符号相同, 余数不需要修正; 否则, 按下列规则修正余数; 当被除数与除数符号相同时, 余数+除数; 否则, 余数减除数



5.4 定点除法运算

5. 补码加减交替法除法

例: 已知 $x = -0.1001$ $y = +0.1101$ 用补码一位除法求 x / y

解: $[x]_{补} = 1.0111$ $[y]_{补} = 0.1101$ $[-y]_{补} = 1.0011$

	被除数/余数	商	说明
	11.0111	0.0000	被除数与除数异号 被除数加除数
+ $[y]_{补}$	<u>00.1101</u>		
	00.0100	0.0000 1	余数与除数同号,商上1,左移减除数
←	00.1000	0.0001	
+ $[-y]_{补}$	<u>11.0011</u>		
	11.1011	0.0001 0	余数与除数异号, 商上0, 左移, 加除数
←	11.0110	0.0010	
+ $[y]_{补}$	<u>00.1101</u>		
	00.0011	0.0010 1	余数与除数同号,商上1,左移 余数减除数



5.4 定点除法运算

5. 补码加减交替法除法

例: 已知 $x = -0.1001$ $y = +0.1101$ 用补码一位除法求 x / y

解: $[x]_{\text{补}} = 1.0111$ $[y]_{\text{补}} = 0.1101$ $[-y]_{\text{补}} = 1.0011$

	被除数/余数	商	说明
	00.0011	0.0010 1	余数与除数同号,商上1,左移 余数减除数
←	00.0110	0.0101	
+ $[-y]_{\text{补}}$	<u>11.0011</u>		
	11.1001	0.0101 0	余数与除数异号, 商上0, 左移,加除数
←	11.0010	0.1010	
+ $[y]_{\text{补}}$	<u>00.1101</u>		
	11.1111	0.1010 0	余数与除数异号, 商上0, 移商
←		1.0100	

商为负, 需要修正, 结果为 1.0101

余数与被除数符号相同, 不需要修正



商 = - 0.1011

余数= - 0.00000001

5.4 定点除法运算

5. 补码加减交替法除法

例: $x = -9$ $y = 2$ 求 $[x / y]_{\text{补}}$

$[x]_{\text{补}} = 10111$

$[y]_{\text{补}} = 00010$ $[-y]_{\text{补}} = 11110$

初始化: 对X进行符号扩展到10位，其中扩展的符号位存于余数寄存器中

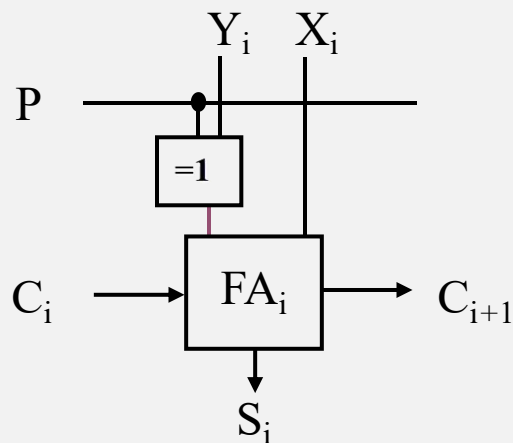
商为负，需修正，结果为 **11110**
余数与被除数符号不同，需修正

被除数与除数符号相异，通过余数-除数修正余数，即: $000001 + 111110 = 111111$

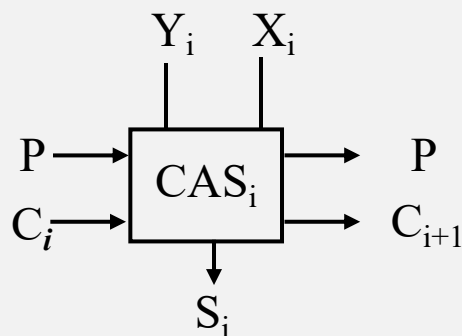
被除数/余数R		上商位 Q_n	说明
	111111	10111	x 与 y 异号, $x+y$
$[Y]$	$+ 000010$		
	000001	10111 1	R 与 Y 同号, 商上1
	$\leftarrow 000011$	0111 1	$R=2R-Y$ (左移加除数)
$[Y]$	$+ 111110$		
	000001	0111 1 1	R 与 Y 同号, 商上1
	$\leftarrow 000010$	111 11	$R=2R-Y$
$[-Y]$	$+ 111110$		
	000000	111 11 1	R 与 Y 同号, $Q_n=1$
	$\leftarrow 000001$	11 111	$R=2R-Y$
$[-Y]$	$+ 111110$		
	111111	11 111 0	R 与 Y 异号, 商上0
	$\leftarrow 111111$	1 1110	$R=2R+Y$
$[Y]$	$+ 000010$		
	000001	1 1110 1	$R>0, Q_n=1$
	$\leftarrow$	1 1101	

5.4 定点除法运算

6. 阵列除法器(电路)



(a) 电路



(b) 符号表示

可控制加/减法(CAS)单元

□ P=0时(全加器)

$$S_i = X_i \oplus (P \oplus Y_i) \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus (P \oplus Y_i)) C_i$$



$$S_i = X_i \oplus Y_i \oplus C_i$$

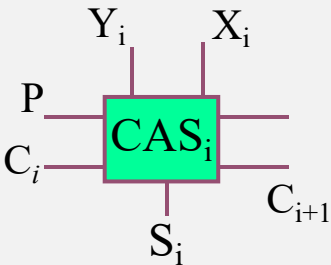
$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

□ P=1时(全减 ---- 还要注意C_i)

$$S_i = X_i \oplus \overline{Y_i} \oplus C_i$$

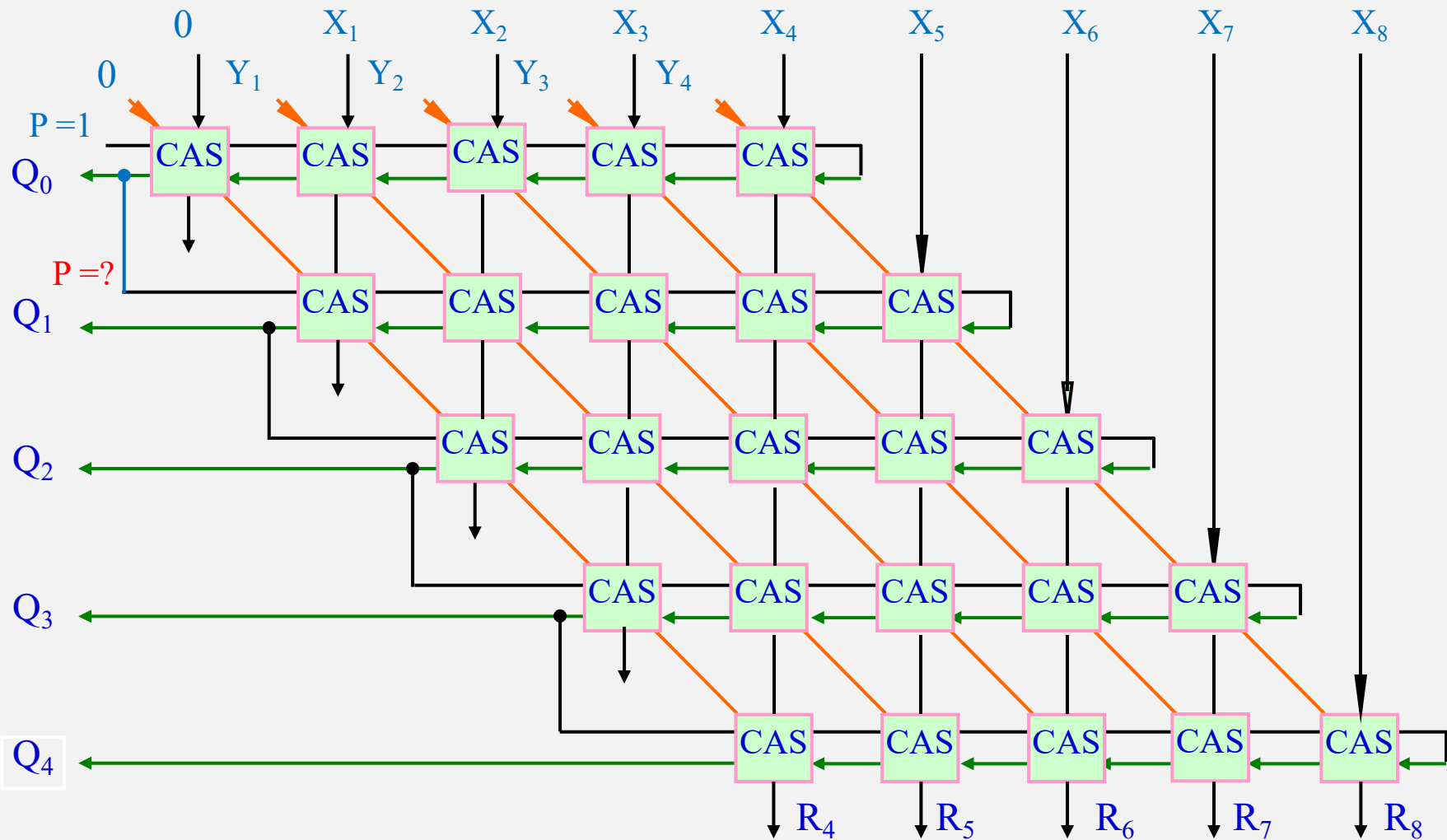
$$C_{i+1} = X_i Y_i + (X_i \oplus \overline{Y_i}) C_i$$

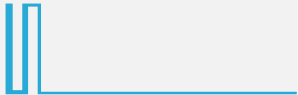
6. 阵列除法器



被除数/余数R		上商位
	001001	
$[-Y]$	+ 110101	
	0 111110	0
	← 111100	
$[Y]$	+ 001011	
	1 000111	01
	← 001110	

$$C_f = Q_n$$





第三部分完